Assignment 2

1

Consider the table schema of LabResult relation:

```
LabResult (lab_id BIGINT, user_id INT, test_name CHAR(20), value FLOAT, unit
CHAR(6), test date DATE)
```

where a `BIGINT` and `INT` take 8 and 4 bytes; a `FLOAT` takes 4 bytes, and a `DATE` takes 4 bytes. A single character takes 1 byte. Assume each data block has a size of 4KB (4096 bytes). Answer the following questions. Ignore block headers, page headers or pointers.

a

What's the size of a record as fixed length record (FLRs)?

✓ **Answer**

$8 + 4 + 20 + 4 + 6 + 4 = 46 \text{ bytes}$

b

How many records fit in one block?

✓ **Answer**

$\left\lfloor \frac{4096}{46} \right\rfloor = 89 \text{ records}$

c

What percentage of space is wasted per data block if FLRs are stored in an unplanned manner in the storage systems (i.e., no record is allowed to be fragmented and stored across two data blocks).

✓ **Answer**

$100\% \cdot (1 - \frac{89 \cdot 46}{4096}) = 0.0488\%$

d

Assume a LabResult table has 100 tuples. How many data blocks are needed to store the table? How many bytes would be "wasted" in this case? – still available but not used to store data?

✓ **Answer**

$\lceil \frac{100}{89} \rceil = 2 \text{ blocks}$

$2 \cdot 4096 - 100 \cdot 46 = 3592 \text{ bytes}$

2

Given the queries Q1-Q4 below, answer the questions.

```
SELECT AVG(value) FROM lab results WHERE test_name='Glucose';
SELECT * FROM lab results WHERE test date = DATE '2025-01-10';
SELECT * FROM lab results WHERE user_id = 123;
SELECT * FROM lab results WHERE LOWER(test_name)='glucose';
```

a

Which queries benefit from an index? For each query, give the SQL that build the index.

✓ **Answer**

Q1-Q3 as their conditionals act as a comparison directly upon a column. We can simply created indexes upon these conditional columns.

```
CREATE INDEX idx_lab_results_test_name ON lab_results(test_name);
CREATE INDEX idx_lab_results_test_date ON lab_results(test_date);
CREATE INDEX idx_lab_results_user_id ON lab_results(user_id);
```

b

Propose one composite index to help Q1 and Q2 together. Give the SQL that create this index.

✓ **Answer**

We can create a composite index upon both `test_name` and `test_date`,

```
CREATE INDEX idx_lab_results_test_name_date ON lab_results(test_name,
test_date);
```

c

Which query does NOT benefit from a standard index and why

✓ **Answer**

Q4, as it filters upon a computed value `LOWER(test_name)`, which in most SQL servers, is not something that is indexable normally.

3

Write SQL for each task using the given schema:

Hint: You may use some short-hand operators such as: Order By, Desc, LIMIT, EXISTS. etc. While there is more than one way to write the queries, try to optimize your SQL efficiently. If you used AI tools such as ChatGPT, you are required to put down the prompts you used and add clarification on how you managed to improve its answer by providing more information to make it to its final form.

a

Average glucose value in 2025.

✓ **Answer**

```sql
SELECT AVG(value) AS avg_glucose_2025
FROM LabResult
WHERE test_name = 'Glucose'
  AND test_date ⩾ '2025-01-01'
  AND test_date < '2026-01-01';
```

b

Top 5 cities by average glucose in 2025.

✓ **Answer**

```sql
SELECT u.city, AVG(l.value) AS avg_glucose_2025
FROM LabResult l
JOIN User u ON l.user_id = u.user_id
WHERE l.test_name = 'Glucose'
  AND l.test_date ⩾ '2025-01-01'
  AND l.test_date < '2026-01-01'
GROUP BY u.city
ORDER BY avg_glucose_2025 DESC
LIMIT 5;
```

c

Users who had BOTH Glucose and Cholesterol tests in 2025.

✓ **Answer**

```sql
SELECT user_id
FROM LabResult
WHERE test_name IN ('Glucose', 'Cholesterol')
```

```
  AND test_date ⩾ '2025-01-01'
  AND test_date < '2026-01-01'
GROUP BY user_id
HAVING COUNT(DISTINCT test_name) = 2;
```

d

For each city, the number of distinct users with any lab test in 2025.

✓ **Answer**

```
SELECT u.city,
       COUNT(DISTINCT l.user_id) AS distinct_users_2025
FROM LabResult l
JOIN User u ON l.user_id = u.user_id
WHERE l.test_date ⩾ '2025-01-01'
  AND l.test_date < '2026-01-01'
GROUP BY u.city
ORDER BY distinct_users_2025 DESC;
```

4

Consider a question “For each city, how many distinct users had any lab test in March 2025?” A student put down a query below:

```
SELECT *
FROM users u
JOIN lab results l ON u.user id = l.user id
WHERE l.test date ⩾ DATE '2025-03-01'
AND l.test date < DATE '2025-04-01';
```

Answer the following:

a

Identify three performance issues or inefficiencies in the query. (Briefly explain each.)

✓ **Answer**

`SELECT *` is typically bad practice as you normally don't need all columns, leading to unnecessary reads.

This query could probably sped up with indexes on `l.user, test`, `u.user`,

It also does not group on city, leading this query to be incorrect for answering the question. You should be selecting on city and count unique IDs too.

b

Rewrite the query to be correct and more efficient for the stated question.

✓ **Answer**

```sql
SELECT u.city,
       COUNT(DISTINCT l.user_id) AS distinct_users_march_2025
FROM lab_results l
JOIN users u ON l.user_id = u.user_id
WHERE l.test_date ⩾ '2025-03-01'
  AND l.test_date <  '2025-04-01'
GROUP BY u.city;
```

c

Propose one index that would help speed up your query and briefly explain why.

✓ **Answer**

```sql
CREATE INDEX idx_lab_testdate_userid
ON lab_results (test_date, user_id);
```

As we are accessing `user_id` and `test_date` fairly frequently when answering these questions, so it would make sense to pre-index these columns.

5

Use an AI tool to generate SQL for the question:
“Find the top 5 users whose lab test values show the largest increase over time in 2025.”

As the designer, you know the schema you have designed, but AI does not know at the beginning.

```
users(user id, name, age, city)
lab_results(lab id, user id, test name, value, test_date)
```

And you can “assume” other conditions: which attributes have an index, any data constraints, the size of the data/tables, data types, etc, that are not revealed to AI yet.

a

Your prompt, and the AI-generated SQL.

✓ **Answer**

Given the following SQL table structure:

users(user id, name, age, city)
lab results(lab id, user id, test name, value, test date)

Find the top 5 users whose lab test values show the largest increase over time in 2025.

```sql
SELECT u.user_id,
       u.name,
       MAX(l.value) - MIN(l.value) AS value_increase
FROM lab_results l
JOIN users u ON l.user_id = u.user_id
WHERE l.test_date ⩾ '2025-01-01'
  AND l.test_date <  '2026-01-01'
GROUP BY u.user_id, u.name
HAVING COUNT(*) > 1
ORDER BY value_increase DESC
LIMIT 5;
```

b

Your revised SQL, that improves correctness, clarity, or performance.

✓ **Answer**

The SQL code looks good to me, other than implying that indexes exist on the test_date and user_id, I don't see any issues

c

A short, 3–5 sentence reflection discussing:

- what the AI did well,
- what was incorrect, inefficient/wrong or unclear, and
- how you guided AI or corrected its output, to improve it.

✓ **Answer**

The AI did very well at actually answering the prompt and creating a good SQL query that answers exactly what the question was.

I did not find the provided SQL to have any issues, so it appears to be okay. The AI did state that this is across all tests assuming the values are comparable, which is a good thing to notice and possibly something to fix, but the question did not specify this.

I did not need to guide to correct the AI in this case.